

Reviewer Pack — Minimal Local Indeterminacy in Categorical Geometry and Comm...

Entry 52b640be5181 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 11:54:42 UTC

Minimal Local Indeterminacy in Categorical Geometry and Communication Complexity Rank

Entry ID: 52b640be5181 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 11:54:42 UTC

1. Conjecture

Field A (mathematical branch): Categorical Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

The minimal local indeterminacy of the categorical structure associated with a given boolean function, as measured by the number of isomorphisms between its categories, is linearly correlated with the communication complexity rank of the function's matrix representation, such that $\text{Minimal Local Indeterminacy}(X) = \Theta(\text{Communication Rank}(X))$ for any X .

Rationale (proposer's reasoning):

Categorical geometry provides a framework for studying structure in abstract algebraic terms. If categorical structures can be used to capture aspects of computational complexity, it could lead to novel insights into communication complexity lower bounds. The conjecture posits that the local indeterminacy of such structures directly relates to the difficulty of communicating information, which is a fundamental concept in communication complexity.

Taxonomy category: categorical_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 86ab30da674dec98

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all boolean functions, the correlation coefficient between Minimal Local Indeterminacy and Communication Rank across at least 25 out of 30 seeds is ≥ 0.8 with a p-value ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - categorical geometry AND communication complexity rank-minimal local indeterminacy IN categorical geometry AND communication complexity in matrices-isomorphisms IN categorical geometry AND correlation with communication complexity rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_rank(matrix):
        n = len(matrix)
        rank = 0
        for i in range(n):
            if any(matrix[i][j] != 0 for j in range(n)):
                rank += 1
                for j in range(n):
                    matrix[j][i] /= matrix[i][i]
                for k in range(n):
                    if k != i:
                        factor = matrix[k][i]
                        for j in range(n):
                            matrix[k][j] -= factor * matrix[i][j]
        return rank

    def minimal_local_indeterminacy(matrix):
        n = len(matrix)
        indeterminacy = 0
        for i in range(n):
            for j in range(i+1, n):
                if all(matrix[i][k] == matrix[j][k] for k in range(n)):
                    indeterminacy += 1
        return indeterminacy
```

```

def generate_boolean_function(n):
    return [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

n_values = [5, 10, 15, 20, 30, 40]
total_indeterminacy = 0
total_rank = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):
        matrix = generate_boolean_function(n)
        indeterminacy = minimal_local_indeterminacy(matrix)
        rank = communication_rank(matrix)
        total_indeterminacy += indeterminacy
        total_rank += rank
        instances_tested += 1

mean_indeterminacy = total_indeterminacy / instances_tested
mean_rank = total_rank / instances_tested

correlation_coefficient = (instances_tested * mean_indeterminacy * mean_rank -
                           sum(ind * rank for ind, rank in zip(range(instances_tested), range(
                               math.sqrt((instances_tested * mean_indeterminacy**2 - sum(ind**2 for
                                   (instances_tested * mean_rank**2 - sum(rank**2 for rank in

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation_coefficient) >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_303fc3fa.py", line 84, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_303fc3fa.py", line 57, in run_trial
    rank = communication_rank(matrix)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_303fc3fa.py", line 28, in communication
    matrix[j][i] /= matrix[i][i]
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14479
2	propose	ollama_remote	glm4:latest	0	0	13022
3	preregistration	ollama_remote	glm4:latest	0	0	9858
4	novelty	ollama_remote	glm4:latest	0	0	12595
5	novelty	ollama_remote	glm4:latest	0	0	9097
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15561
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12578
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11802
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10230
10	judge	ollama_remote	glm4:latest	0	0	11621

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120843 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/52b640be5181.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/52b640be5181.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/52b640be5181.tar.gz` (if generated)